

Betriebssysteme

2 – ÜB – Prozessverwaltung

Aufgabe 1: Grundkonzepte

a) Ein Kollege argumentiert: *"Dass ein Browser für jeden offenen Tab einen eigenen Prozess anlegt, ist gar nicht so gut: dann kann nämlich nur ein einzelner Tab etwas tun (z. B. Musik wiedergeben)."* Der Kollege hat nicht Recht. Warum? Geben Sie zudem an, was für Vorteile und Nachteile ein Prozess pro Tab wirklich hat.

b) Kann ein Rechner mit einem Kern mehrere Programme so ausführen, dass ein Endnutzer mit diesen Programmen gleichzeitig arbeiten kann? Falls ja: Wie? Falls nein: Warum nicht?

Hinweis: Sie dürfen hier gerne Wortklauberei betreiben!

c) Was ist im Rahmen eines Prozesswechsels konkret für das Betriebssystem zu tun?

d) Wie muss eine Anwendung einen Prozesswechsel im Code berücksichtigen?

Hinweis: Woran merkt eine Anwendung, dass der zugehörige Prozess oder ein Thread vom Betriebssystem verdrängt wird oder auf blockiert schaltet?

e) Geben Sie drei (konzeptuell unterschiedliche) Gründe dafür an, dass ein Thread vom Aktiv-Zustand in den Blockiert-Zustand wechselt.

f) Was ist damit gemeint, dass ein Thread von der CPU *verdrängt* wird? Warum landet der Thread dann im Bereit- statt im Aktiv-Zustand?

g) Warum braucht jeder Thread einen eigenen Stapel?

h) Angenommen, es gibt mehr Kerne als Threads. Warum gibt es dann keine Threads, die (mehr als sehr kurzfristig) im Bereit-Zustand sind?

i) Warum führt die Bevorzugung kurzer Prozesse zu einer kürzeren mittleren Bearbeitungszeit sowie zu geringerer Auslastung? Welche Annahmen stecken in dieser Argumentation?

j) Probieren Sie die Codebeispiele zu Threads (F26, F27, F30, F31, F32, F33, F35, F40) auf einem echten System aus! Achten Sie besonders darauf, ob die genannten Fehler überhaupt und konsistent zu reproduzieren sind.

k) Liefert First-Come-First-Serve eine eindeutige Planungsreihenfolge? Argumentieren Sie an einem Beispiel.

l) Gegeben seien die folgenden Prozesse mit [A]ktiv- und [B]lockier-Zeiten (z. B. in Sekunden angegeben):

- Prozess 1: A3, B3, A3, B3, A1
- Prozess 2: A2, B4, A2
- Prozess 3: A5, B1, A5, B1, A1

Die Deadline für Prozess 1 ist 15, für Prozess 2 ist die Deadline 14, und für den Prozess 3 ist die Deadline 16.

Führen Sie eine Planung mit dem Earliest-Deadline-First-Verfahren durch und zeichnen Sie das Ergebnis. Werden alle Deadlines eingehalten?

m) Warum sind im vorangehenden Beispiel keine Bereit-Zeiten angegeben?

n) Betrachten Sie Race Conditions und Deadlocks für Prozesse und Dateien:

- Gehen Sie davon aus, dass das OS von einem Prozess geöffnete Dateien nicht für andere Prozesse sperrt. Was müssen Prozesse machen, dass es eine Race Condition gibt und wie äußert sich die Race Condition?
- Gehen Sie davon aus, dass das OS von einem Prozess geöffnete Dateien für andere Prozesse sperrt. Löst das das Problem aus dem vorigen Punkt? Wieso und was passiert?
- Gehen Sie immer noch davon aus, dass das OS geöffnete Dateien sperrt. Wie kann es in dieser Situation zu einem Deadlock kommen? Was muss welcher Prozess in welcher Reihenfolge tun?

o) Wieso ist das Scheduling dafür verantwortlich, dass Race Conditions schwierig zu reproduzieren sind? Was muss man am Scheduling (und gegebenenfalls an den Prozessen) ändern, damit Race Conditions sicher reproduzierbar werden?

p) Erklären Sie für jedes der 4 Deadlock-Kriterien, warum kein Deadlock mehr möglich ist, wenn das Kriterium nicht mehr erfüllt ist (aber alle anderen Kriterien weiterhin erfüllt sind).

- q) Bei einem Lese-Schreiber-Problem versuchen mehrere Leser und Schreiber, auf die selbe Variable zuzugreifen. Geben Sie die Einschränkungen wieder, die bei Leser-/Schreiber-Problemen für den Zugriff gelten (Wann darf ein Schreiber schreiben? Wann darf ein Leser lesen?).

Beschreiben Sie für Leser und Schreiber jeweils das Problem, das beim Verzicht auf diese Einschränkungen entstehen kann.

- r) Die Abfrage der Member-Variable `value_` hat einen Bug. Welchen? Wann und wie äußert sich ein Problem in diesem Zusammenhang?

```
float get_value()
{
    m_.lock();                // für einen Mutex m_
    return value_;
}
```

- s) Schreiben Sie eine Anwendung in einer Sprache Ihrer Wahl für folgendes Szenario: Zwei Threads füllen eine FIFO-Schlange mit Strings der Art "Thread [Threadnummer] -- [Fortlaufende Zahl pro Thread]", z. B. "Thread 2 - 4", "Thread 2 -- 5", "Thread 1 -- 3", "Thread 2 -- 6", und so weiter. Drei Threads lesen die Strings aus, entfernen sie aus der Schlange und geben sie auf der Kommandozeile aus.

Hinweise:

- Es darf weder Race Conditions noch Deadlocks geben.
- Nehmen Sie das Mutexing selbst vor.
- Achtung: bei manchen Sprachen ist die FIFO-Schlange selbst schon thread-sicher. In diesem Fall äußern sich Fehler beim Mutexing womöglich nicht.
- In der Praxis treten solche *Producer-Consumer-Szenarien* häufig auf, z. B. wenn ein Sensor in einem selbstfahrenden Auto Hindernisse meldet, die von der Steuerungslogik abgearbeitet werden müssen.

- t) Angenommen, die Producer-Threads in der vorangehenden Teilaufgabe erzeugen sehr schnell Strings, während die Consumer-Threads sehr langsam Strings aus der Queue entnehmen.

Was für ein Problem ergibt sich? Ist das ein Deadlock? Wie lässt sich das Problem in der Praxis lösen?

Hinweis: Sie können das Problem im Code der vorigen Aufgabe nachstellen, indem Sie die Consumer-Threads nach jedem Verbrauchen mit einer `sleep`-Anweisung für eine hinreichend lange Zeit warten lassen.

Aufgabe 2: Philosophen-Problem

Die folgende Aufgabe ist auch als Philosophen-Problem bekannt:

Vier Philosophen treffen sich, um ihren wichtigsten Tätigkeiten nachzugehen: Essen und denken. Da sie aber etwas zerstreut sind, vergisst jeder eines seiner zwei Essstäbchen zu Hause. Die Küche bringt zwar jedem der Philosophen einen ständig vollen Teller Reis, davon essen kann ein Philosoph jedoch nur, wenn er zwei Stäbchen zur Verfügung hat. Also einigen sie sich darauf, dass jeder sein Stäbchen auf seine rechte Seite legt. Da sie an einem runden Tisch sitzen, hat so jeder von ihnen zwei Stäbchen neben sich liegen. Es werden folgende Regeln ausgemacht:

- Ein Philosoph, der Hunger hat, nimmt sich erst sein linkes Stäbchen, dann sein rechtes, und isst, bis er satt ist. Danach legt er beide Stäbchen wieder hin und denkt, bis er wieder hungrig wird.
- Das Aufheben eines Stäbchens erfolgt mit der gebotenen Höflichkeit: Exakt ein Philosoph greift zu einer bestimmten Zeit nach einem Stäbchen, alle anderen halten sich zurück und bekommen das Stäbchen nicht.
- Ein einmal aufgenommenes Stäbchen gibt man nicht wieder her, bis man sich vorest satt gegessen hat.

a) Nach drei Wochen sind alle Philosophen verhungert. Warum? Stellen Sie hierzu den Zustand der Philosophen vor dem Verhungern als Betriebsmittelgraph nach dem Diagramm der Vorlesung dar. Erläutern Sie, welche der vier Bedingungen für eine Verklemmungen erfüllt werden und warum.

b) Nach dem Debakel aus Teil a) werden neue Philosophen eingesetzt. Sie wollen verhindern, dass auch diese verhungern, und haben folgende Optionen:

- Einer der neuen Philosophen ist so geschickt, dass er beide Essstäbchen gleichzeitig greifen kann (und das nur tut, wenn auch beide gerade daliegen).
- Sie setzen einen Philosophen in die Runde, der– wenn er länger als eine Stunde auf ein Stäbchen wartet – ein eventuell erworbenes Stäbchen zurücklegt und kurz mit seinem Hunger lebt, ehe er es neu versucht.
- Sie setzen einen Assistenten ein, der alle Stäbchen verwaltet und sie auf Anfrage paarweise an die Philosophen ausgibt.
- Sie wählen diesmal nur Philosophen einer konkurrierenden Denkschule: Diese nehmen sich immer zuerst das rechte Stäbchen, dann das linke.
- Einer der Philosophen lebt nach dem Motto „mens sana in corpore sano“ und ist so kräftig, dass er seine Nachbarinnen zur Freigabe ihrer Stäbchen zwingen kann. Nach Essen und Rückgabe der Stäbchen geht er stets erst mal eine Runde joggen.

Bei welchen dieser Optionen läuft die Runde wieder Gefahr zu verhungern (und warum)? Falls nicht, welche der Bedingungen für Verklemmungen sind nun nicht mehr erfüllt? Legen Sie (wie immer) Annahmen ausreichend dar und formulieren Sie präzise!

Aufgabe 3: Ex-Klausuraufgabe

In der Finanzbuchhaltung graueze11en arbeiten drei Bürokräfte *Alice* (A), *Bob* (B) und *Cheryl* (C) gleichzeitig an Unternehmensdokumenten. Konkret müssen regelmäßig zwei besonders wichtige Dokumente bearbeitet werden: die *Personalliste* (P) und der *Quartalsbericht* (Q). Beide Dokumente sind Dateien aus einer handelsüblichen Bürosoftware (z. B. Excel-Dateien). Jede Bürokraft muss hin und wieder jedes Dokument bearbeiten.

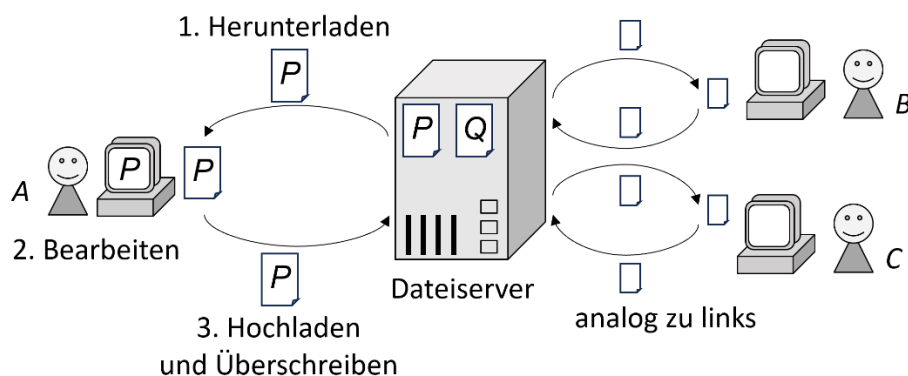
Die Dokumente liegen im aktuellen Stand auf einem Dateiserver, der über das Intranet des Unternehmens zu erreichen ist. Auf dem Dateiserver können die Dokumente aber nicht direkt bearbeitet werden, weil die Bürosoftware dort nicht läuft.

Zur Bearbeitung geht jede Bürokraft daher nach dem Vorgehen V_1 vor:

1. Die Bürokraft verbindet sich mit dem Server und lädt den aktuellen Stand des Dokuments vom Dateiserver auf den eigenen Bürorechner herunter.
2. Die Bürokraft trennt die Verbindung zum Dateiserver und bearbeitet im Anschluss das Dokument lokal auf dem eigenen Rechner.
3. Die Bürokraft verbindet sich zum Server und lädt die fertig bearbeitete Datei dort hoch. Die bearbeitete Datei überschreibt die vorherige Version.

Insgesamt können die Bürokräfte damit zeitgleich die Dateien bearbeiten, z. B. kann A die Datei P bearbeiten, während B die Datei Q bearbeitet.

Der Gesamtzusammenhang ist in folgender Abbildung illustriert:



Hinweise:

- Dass A im Diagramm P bearbeitet, ist nicht zwingend so, sondern nur ein Beispiel. Genauso gut könnte A die Datei Q bearbeiten.
- Da das **reine** Herunterladen, Hochladen und Überschreiben äußerst wenig Zeit braucht (wenige Millisekunden), sei angenommen, dass sich diese Vorgänge **nie** zeitlich überlappen. Es wird z. B. **niemals** zugleich eine Datei heruntergeladen und überschrieben. Gleichzeitiges Bearbeiten ist aber möglich.

- Das Intranet ist zuverlässig genug, dass alle Hochlade- und Herunterlade-Vorgänge auf jeden Fall gelingen. Es gibt keine Verbindungsabbrüche. Der Server ist ebenfalls so zuverlässig, dass alle Dateioperationen immer gelingen.

a) Im praktischen Betrieb merkt man schnell, dass das obige Vorgehen V_1 ein großes Fehlerpotential birgt. **Erklären Sie** mit Ihrer Kenntnis der Betriebssysteme an einer konkreten Abfolge von Hoch- und Herunterlade-Vorgängen der Bürokräfte A , B und C , **was schief gehen kann**, und **geben Sie ein Fachwort für das Problem an**.

b) Um das Problem zu lösen, schlägt die IT ein komplexeres Vorgehen V_2 vor:

1. Sobald ein Dokument heruntergeladen wird, markiert der Dateiserver das Dokument als "in Bearbeitung". Ein so markiertes Dokument kann zunächst von keiner weiteren Bürokraft mehr heruntergeladen werden. Versuche ergeben den Hinweis, zu warten, bis die Bearbeitung abgeschlossen ist.
2. Die Bürokraft, welche das Dokument heruntergeladen hat, bearbeitet **ausschließlich** jenes Dokument am Rechner fertig und lädt das Ergebnis hoch. Dieser Gesamtvorgang solle als Annahme **immer** irgendwann fertig sein.
3. Nach dem Hochladen wird das serverseitige Dokument automatisch ersetzt, die Kopie auf dem Rechner der Bürokraft automatisch gelöscht und zuletzt die Markierung "in Bearbeitung" automatisch entfernt.

Geben Sie **ein Fachwort** aus den Betriebssystemen an, welches diese Realisierung beschreibt. **Löst das Vorgehen V_2 das Problem von zuvor?** Begründen Sie!

c) Nehmen Sie an, dass das komplexere Vorgehen V_2 der IT wie erklärt umgesetzt wird. Dummerweise zeigt sich jetzt eine andere Sorte Missverhalten:

Angenommen, Bürokräfte können beim Ändern einer beliebigen Datei bemerken, dass sie die andere Datei mitziehen müssen (z. B. um Verweise zu aktualisieren). Die entsprechende Bürokraft lädt in diesem Fall die zweite Datei vom Dateiserver ebenfalls herunter. Erst wenn die Änderungen an **beiden Dateien** abgeschlossen sind, darf die Bürokraft beide Dateien wieder auf den Dateiserver hochladen.

Was geht jetzt – als **Fachbegriff** der Betriebssysteme – schief? **Wie äußert sich das Problem? Erklären Sie** an einer konkreten Folge von Herunter- und Hochlade-Vorgängen der Bürokräfte A , B und C .

d) Nehmen Sie weiterhin die komplexere Realisierung der IT nach V_2 an. Nehmen Sie zudem an, dass die Bearbeitung von Datei P nie den Anlass gibt, auch Datei Q zu bearbeiten. Es kann aber jeder Bürokraft passieren, dass bei der Bearbeitung von Datei Q die Datei P ebenfalls nachgezogen werden muss. In diesem Fall lädt die Bürokraft erst Q , dann P herunter, abschließend beide Dateien gemeinsam hoch. **Kann jetzt etwas schiefgehen?** Begründen Sie mit einer **Bedingung** der VL.